Shor in Miniature — Challenge 2

Overview

Teams factor small numbers with Shor's algorithm on a laptop simulator (Part A), then argue with real numbers how far today's machines are from breaking RSA-2048 (Part B). The challenge punctures both the hype ("quantum computers break RSA now") and the complacency ("it will never happen").

Learning goals: quantum phase estimation, modular arithmetic as reversible circuits, continued fractions, why a correct implementation must not use the answer, and how error correction turns a few thousand logical qubits into hundreds of thousands of physical ones.

Format: teams of 3–4, 24 hours. Part A is scored automatically against hidden test numbers; Part B is a 2-page written estimate judged by rubric; both are presented in 5 minutes.

Prerequisites: linear algebra, Python, and an introduction to qubits and gates. A 60-minute opening lecture covers the reduction from factoring to order finding, phase estimation, and continued fractions.

Tooling: any statevector simulator (Qiskit Aer, Cirq, PennyLane). A laptop with 8 GB RAM is enough for every required task.

Part A: factor with Shor

Write a function `build_shor(N, a)` that returns the order-finding circuit for any odd composite N up to 6 bits and any a coprime to N, plus a classical driver `factor(N)` that picks a, runs the circuit, recovers the order r with continued fractions, and returns the factors.

The interface is the rule. The generator receives only N and a. It must not receive or compute the order r, and it must work unchanged on numbers the team has never seen. Judges call it on hidden test numbers.

Levels (each level includes the previous):

Level	What teams build	Qubits for N = 21	Points available
1	Controlled modular multiplication written as permutation matrices; counting register of 2n qubits	15	40
2	Modular multiplication built from gates (QFT-based adders, as in Beauregard 2003)	22	+30

On a laptop, Level 1 for N = 21 runs in seconds and Level 2 in a few minutes. The full-register Level 2 circuit for 6-bit numbers needs 26 qubits (1 GB statevector) and may take too long; the bonus version handles it in 15 qubits.

Deliverables: a repository with `build_shor`, `factor` and a README; a results table for N = 15 and 21 showing outcome histograms and the recovered order for each a tried.

Forbidden:

- Hard-coding r, the factors, or which work-register states occur ("compiled Shor"; see Smolin, Smith & Vargo, Nature 2013).
- Choosing a or circuit structure by looking up N in a table.
- Replacing the quantum circuit with classical order finding anywhere in `factor`. Classical precomputation of the constants a^(2^j) mod N is allowed and expected.

Part B: the RSA-2048 reality check

Teams write a 2-page estimate answering one question: how far is today's best hardware from factoring a 2048-bit RSA key, and what has to improve? They must cite sources and show their own arithmetic.

Published estimates teams should find and explain:

Year	Source	Target	Physical qubits	Runtime	Key assumptions
2026	Google Quantum AI whitepaper (March)	ECC-256	under 500,000	about 9 minutes after the public key is known	Circuits withheld; estimate verifiable by zero-knowledge proof
2026	Iceberg Quantum, Pinnacle architecture (February)	RSA-2048	under 100,000	not stated	QLDPC codes; needs longer-range connectivity; simulation only
2025	Gidney, arXiv:2505.15917	RSA-2048	under 1,000,000	under 1 week	0.1% gate error, 1 µs cycle, 10 µs reaction time, nearest-neighbour grid
2019	Gidney &	RSA-2048	about	about 8 hours	Same hardware

Year	Source	Target	Physical qubits	Runtime	Key assumptions
	Ekerå		20,000,000		assumptions as the 2025 paper

The 2019→2025 drop of 20× came from algorithms, not hardware: the assumptions were unchanged.

Required tasks:

1. Explain why the estimates trade qubits against runtime (the 2025 paper uses 20× fewer qubits but runs about 20× longer).
2. Build a toy surface-code model and reproduce the order of magnitude. Use a logical error rate per qubit per cycle of

$$p_L \approx 0.1 \left(\frac{p}{p_{th}} \right)^{(d+1)/2}, \qquad p_{th} \approx 1\%, \qquad \text{physical qubits per logical qubit} \approx 2d^2$$

   and choose the code distance d so that L logical qubits running for T cycles fail with probability below 10%: L · T · p_L < 0.1.
3. Compare with the largest machines announced today (qubit count and two-qubit error rate) and state the gap as a factor.
4. Give a timeline argument with explicit assumptions, and one consequence for organisations (harvest-now-decrypt-later; why migration to ML-KEM starts now).

Worked example for judges (illustrative inputs, not a published figure): L = 1,500 logical qubits, 5 days at 1 µs per cycle gives T ≈ 4.3×10^{11} cycles. With p = 0.1%, the condition requires d = 29, about 1,700 physical qubits per logical qubit, so about 2.5 million in total before counting magic-state factories. A team that lands within the 10^6–10^7 range and can explain why Gidney's tricks (yoked surface codes, magic state cultivation, approximate residue arithmetic) bring it under one million has done the task well.

Sources: Gidney 2025 abstract; The Quantum Insider, "Q-Day Just Got Closer", 31 March 2026 for the 2026 papers.

Scoring

Part A is 85 points (automatic), Part B 40 points (rubric) and the presentation 15 points, for 140 in total.

Part A: two automatic checks per hidden test case. Checking only the final factors is not enough: the classical verification step means even random measurement outcomes sometimes yield factors. The judge therefore also checks that the circuit produces the right distribution.

1. **Distribution test:** total variation distance between the team's histogram (2,000 shots) and the ideal distribution must be below 0.1.
2. **Factoring test:** `factor(N)` returns the correct factors within the time limit, using only the team's circuit.

Level	Requirement	Points
1	Both tests pass for all 4-bit and 5-bit cases	40
2	Gate-level construction passes both tests for (15, 7) and (21, 2)	30

A level counts only if the one before it passes. Any evidence of compiled Shor or a lookup table voids Part A for that team.

Part B rubric (40):

Criterion	Points
Correct, cited summary of published estimates and their assumptions	10
Own surface-code calculation, arithmetic shown, result in the right order of magnitude	12
Honest comparison with current hardware, gap stated as a factor	8
Timeline argument with explicit assumptions and a migration recommendation	10

Presentation (15): clarity of the order-finding explanation (5), one thing that went wrong and how they found it (5), answering judges' questions (5).

Tie-breakers: fewer qubits on the hidden 6-bit case, then lower total gate count.

Hints and common mistakes

Release hints in levels so stuck teams keep moving.

Level 1, in the starter kit:

- For N = 21 and a = 2 the order is 6. Use it only to check your output; your code must not know it.
- The counting register needs 2n qubits so that $2^{(2n)} \geq N^2$; with fewer, continued fractions can return the wrong denominator.
- One shot often gives a divisor of r, not r itself. Combine denominators from several shots with the least common multiple and check $a^r \bmod N = 1$.

Level 2, at 12:00:

- Test each arithmetic block on its own against classical arithmetic for every input before assembling the full circuit.
- Check your QFT convention (sign and bit order) against your simulator's documentation; most "almost right" histograms are a reversed register.

- The controlled multiplier must be undone with the multiplier for the modular inverse of a, or the ancillas stay entangled and the peaks blur.

Mistakes mentors should watch for:

Mistake	Symptom	Fix
Counting register measured in reversed bit order	Peaks at wrong positions; distribution test fails	Reverse the bitstring or the register
Work register not initialised to 1	Flat or single-peak histogram	Prepare the work register in state 1
Continued fractions allowed denominators ≥ N	Huge, wrong orders	Limit the denominator to below N
No check that gcd(a, N) = 1	Crash or nonsense on the shared-factor test case	Check classically first; a shared factor already factors N
Order found but a^(r/2) ≡ −1 mod N	Only trivial factors	Retry with a new a
Circuit built for one N only	Fails every hidden 6-bit case	Generate everything from N and a

Keep the reference solution private until prizes are awarded, then publish it with the hidden test set.